Human-in-the-Loop Engineering

Designing Systems Where Agents Propose, Humans Approve

3 June 2026 · Sofia · 2-hour workshop

Stefan Angelov · Lyubomir Bozhinov

Your guides today

Stefan Angelov

Architect Lead · Engineering Manager

"The Java guy from pLoVEdiv"

@cefothe

Lyubomir Bozhinov

Technology Leader

Bridging Strategy, Innovation & Scalable Execution

@lyubomir-bozhinov

What this workshop is

This is not a talk you sit and watch. You sit **with us** while an AI coding agent builds a real service from a real PRD — and we stop the agent at every moment that matters.

Every stop is a decision we make as a room. Where should events go? What do we do about this contradiction? Should we accept this estimate? We don't move on until we've decided together.

The PRD is deliberately flawed — the same flaws you see every week. The agent is genuinely capable. The question is: **capable of what, decided by whom?**

What you walk away with

- A filled-in `CLAUDE.md` for a Java webhook-receiver project — yours to keep and adapt
- A partially-built, running receiver that demonstrates the architecture in action
- A working mental model for where HITL checkpoints belong in your own projects

> *Not a finished product. The point is the process.*

The scenario

Maria K. is a Payments PM at a fictional company. She wrote a PRD asking engineering to fix a broken webhook integration with payment provider **PayHub**.

The PRD has every real flaw: silent assumptions, internal contradictions, deferred decisions, an optimistic timeline. It is a normal PRD.

We hand it to Claude Code and step in when human judgment is required — not to fix bad writing, but because some decisions belong to the room, not the agent.

Architecture — pre-decided

Path	Stack	Reason
Hot path	Pure Java 21, no framework	Sub-5s ACK, sustained throughput
Warm path	Spring Boot 3.x	Processing, retries, persistence — dev velocity dominates
Cold path	Spring Boot 3.x	Admin, reconciliation, DLQ

Left alone, Claude Code would default to Spring Boot everywhere. The first thing `CLAUDE.md` does is forbid that default on the hot path.

Agenda — 2 hours

Time	Block
0:00 – 0:05	Welcome
0:05 – 0:20	PRD walkthrough
0:20 – 0:55	Build `CLAUDE.md`
0:55 – 1:40	Drive Claude Code
1:40 – 1:55	Wrap-up
1:55 – 2:00	Buffer

We will **NOT** finish the implementation. We **WILL** finish the `CLAUDE.md`. That is the priority.

HITL beats — what we decide together

- Where to **store events**
- How to **map identities** between PayHub and the local system
- What to do about an **internal contradiction** in the PRD
- A **major architectural decision** the PRD doesn't mention
- Whether to accept the agent's **effort estimate**

> *Not staged. They emerge from the PRD.*

The Cast

8 specialized agents

Each refusing to silently decide

Why a cast, not a single agent?

A general-purpose agent silently makes decisions that should never be made silently: which database, what SLA, whether the feature is ready to ship, what counts as tested. Those decisions belong to humans — or at minimum, in front of humans before anything is committed.

Splitting work across role-shaped agents doesn't make the system safer by making it slower. It makes it more honest by giving each agent explicit boundaries, escalation rules, and a documented refusal to override the humans who own the decision.

> *The agent doesn't get less capable. The **system** gets more honest.*

The roster

`product-owner`
Stewards the PRD. Surfaces silent assumptions. Won't let engineering quietly resolve open product questions.

`project-manager`
Owns scope, schedule, risk. Pushes back on unrealistic timelines. Runs HITL checkpoints.

`solution-architect`
Architecture, ADRs, trade-offs. Refuses to make architecturally significant decisions silently.

`backend-developer`
JVM-first implementation. Java 21, Spring Boot 3.x, virtual threads, persistence, observability.

`qa-lead`
Test strategy, quality gates, release criteria. Won't declare quality "verified" without evidence.

`qa-engineer`
Hands-on test design and automation across functional, integration, perf, security, accessibility.

`code-reviewer`
Pre-merge correctness, security, OWASP, rule enforcement. Last reasonable check before merge.

`devops-engineer`
CI/CD, GitOps, Kubernetes, IaC, observability. Owns the production runtime story.

What they all share

- **Documented refusal points** — things each agent won't decide alone
- **Explicit escalation paths** — to the architect, the PO, the room
- **Project rules are binding** — `CLAUDE.md` wins; agents flag conflicts, never override
- **Severity discipline** — Critical · Major · Minor · Suggestion, never conflated
- **Evidence over taste** — every finding cites file, line, rationale

> *How HITL discipline survives contact with velocity pressure.*

The Pipeline

PRD → epic → stories → done

A human gate on every arrow

The flow

Step	Command	Writes	Gate
1	`/prd:review`	analysis	classify open Qs: blocking / deferrable
2	`/prd:to-epic`	epic	architecture decisions (data store, ordering)
3	`/epic:decompose`	stories	identity mapping, assumed ACs
4	`/epic:estimate`	estimate	accept / cut scope / move date

> *All artifacts land in `work/` as plain markdown. Diffable in a PR. Take-home after the workshop.*

The four-step macro

Every gate, every time:

1. **DETECT** — name the boundary. *"We're about to choose the data store."*
2. **PROPOSE** — 2–3 concrete options with trade-offs. Never one option dressed as a question.
3. **DECIDE — BLOCK** — `AskUserQuestion`. The command halts. No *"I'll assume X for now."*
4. **RECORD** — ADR in `work/decisions/NNNN-*.md`. Linked from the artifact's `hitl:` list.

> *The **command** owns the BLOCK. Agents are advisory — they cannot put a question to the room, and they cannot write the artifact and skip the stop.*

Two loops, one protocol

Loop 1 — Decomposition

*Things that block **planning**.*

`/prd:to-epic` · `/epic:decompose`

→ data store, retention period, identity mapping.

Loop 2 — Implementation

*Things visible **only in the code**.*

`/story:start`

→ "this slice assumes X — true?" Invisible until the developer opens the story.

A story's `hitl:` list growing *after* `/story:start` is the visible proof loop 2 fired.

The Toolbelt

Making the agent context-efficient, deterministic, and persistent

Deterministic repo context

`cxpak`

Spends CPU cycles so you don't spend tokens. The LLM gets a briefing packet instead of a flashlight in a dark room.

→ Deterministic, reviewable repo context.

`github.com/Barnett-Studios/cxpak`

`codeindex`

Local-first codebase index engine. Builds a semantic index on your machine — no code leaves the box.

→ Retrieval layer for grounded answers on large repos.

`github.com/askcodebase/codeindex`

Token optimization — `rtk`

Rust Token Killer. A single Rust binary installed as a Claude Code `PreToolUse` hook.

Transparently compresses noisy Bash output **before** it reaches the model:
`cargo test` · `git status` · `find` · `grep` · `npm` · `docker` · `kubectl`

> `cargo test` *with 262 passing tests:* ***4,823 → 11 tokens.***
> *Average savings across 2,900+ commands:* ***~89%.***

Why HITL cares: longer sessions, cheaper iterations, less context pollution between human review cycles.

`github.com/rtk-ai/rtk`

Session memory — two takes

`claude-mem` — auto-capture

Records the session, compresses with Agent SDK, auto-injects context on restart. Works across Claude Code, Codex, Gemini CLI, Windsurf, OpenCode, Copilot.

→ *"I don't want to re-prime every morning."*

`github.com/thedotmack/claude-mem`

`memory-palace` — deliberate, MCP-based

Explicit `remember` · `recall` · `forget` tools over a knowledge graph with semantic search.

→ *"The human chooses what the agent knows long-term."*

`github.com/jeffpierce/memory-palace`

Choosing memory strategy is itself HITL

You want…	Pick
Frictionless continuity	`claude-mem` — auto
Auditable, curated knowledge	`memory-palace` — manual + MCP
Neither — every session clean	The `CLAUDE.md` itself

> *The discipline is not picking the fanciest tool. It is being explicit about what the agent assumes between sessions.*

What you need to follow along

- Java 21+ installed (`java --version` shows 21+)
- Maven 3.9+ or Gradle
- **Claude Code** (Anthropic's CLI) installed and signed in
- An Anthropic API key — we distribute one in the room
- This repo cloned locally

Or just watch. Repo is public — replay everything after.

`github.com/cefothe/hitl-engineering-jprime-2026-workshop`

After the workshop

The `CLAUDE.md` you walk away with is **yours**. The same shape works for your projects:

1. Decide what the agent should **assume** — architecture, conventions, forbidden patterns
2. Decide what it should **ask** — the HITL checkpoints
3. Decide what it should **refuse** — the boundaries

> *The artifact is small. The discipline is the point.*

Let's begin.

Open the PRD. Meet Maria.

Questions during, not just at the end. Interrupt us.

#jprime2026 · @cefothe · @lyubomir-bozhinov